深度研究报告

CCH 运行时网络故障与部署加固报告

错误 DNS 覆盖、入口/出口链路混淆，以及跨平台 restart/probe 修复

Codex

2026-03-25

本报告无外部封面图，主体证据来自运行时配置、容器日志和本地命令输出。

核心问题：这次 RC/Hiyo 直连异常究竟坏在哪一层；为什么 Tailscale Serve 正常并不能推出容器可以出公网；以后如何更快定位和恢复？

研究范围：运行时目录 `/home/fanghaotian/Applications/claude-code-hub`，仓库 `/home/fanghaotian/Desktop/src/cch/claude-code-hub`，证据窗口截至 2026-03-25 20:55 CST。

目标读者：claude-code-hub 维护者、部署脚本维护者、日常运维排障者

证据窗口：本地运行配置、容器日志、健康检查、连通性探测，以及仓库当前工作树，访问日期均为 2026-03-25

目录

1 执行摘要 **2**

2 研究范围与方法 **2**

2.1 证据边界 2

2.2 时间表示 3

3 这次 Bug 到底坏在哪里 **3**

3.1 先排除一个常见误判：provider 没有被选错 3

3.2 最强根因：runtime compose 中的坏 DNS 覆盖 3

3.3 修复后的 live 容器已经不再携带该 DNS 覆盖 4

4 为什么 Tailscale Serve 正常，不代表容器能出公网 **4**

5 当前状态的精确结论 **5**

5.1 `right.codes` 现在对 host 和容器都正常 5

5.2 `codex.hiyo.top` 的现状是 endpoint 级残余，不是同一根因 5

5.3 关于 Docker NAT / `DOCKER` 链 6

6 部署脚本加固了什么 **6**

6.1 bash 与 PowerShell 现在都支持 runtime 模式 6

6.2 新增能力总表 6

6.3 README 和测试也同步了 7

7 以后怎么修 **7**

7.1 推荐的排障顺序 7

7.2 推荐命令 8

7.3 真正要避免的不是“重启”，而是“盲重启” 8

8 结论与后续问题 **8**

8.1 已被强证据支持的结论 8

8.2 大概率成立，但证据弱于主结论的判断 9

8.3 仍待继续观察的问题 9

A 来源清单 **9**

1 执行摘要

核心结论

这次 “RC/Hiyo 不能直连” 的主因并不是 provider 的 `base URL` 或选择逻辑被改坏，而是 **运行时 compose 给容器加了错误的硬编码 DNS**。保留下来的最强证据是备份文件 `/home/fanghaotian/Applications/claude-code-hub/docker-compose.archbox.yaml.bak-20260325-1912` 中的 `dns: 119.29.29.29 / 223.5.5.5`。这会把问题从 “应用配置层” 推到 “容器出公网与解析链路层”。

围绕这次事故，可以得到四个比 “把配置改爆了” 更精确的判断：

- **provider 选择本身没有坏**。日志里两次成功请求都明确显示选中的仍然是 `RC`，而且 endpoint 仍然是 `https://right.codes/codex/`。
- **Tailscale Serve 正常不等于容器能出公网**。前者证明的是别的机器可以通过 tailnet 打到宿主机上的服务入口；后者要求容器能主动完成 DNS 解析、TCP/TLS 建连并访问外部站点，二者是不同方向的链路。
- **当前已修复的部分是容器网络回到正常形态**。现在 live 的 `claude-code-hub-app-s6gz` 没有 Docker 级别 `Dns` 覆盖，健康检查正常，`right.codes` 从 host 和容器都能访问。
- **当前剩余的 Hiyo 症状并不等价于原事故**。 `codex.hiyo.top` 现在在 host 和容器里都会报同样的 TLS EOF，这更像 endpoint 侧问题，而不是 CCH 或 Docker 继续坏着。

交付结果

除现场修复外，仓库中的部署脚本也已经加固：`scripts/deploy.sh` 与 `scripts/deploy.ps1` 新增了跨平台的 `restart` / `probe-only` 模式，能自动打印 compose 状态、容器 health、端口映射、宿主机健康探测、容器内健康探测，并在失败时给出最近日志；对应 bash 路径已有单元测试覆盖。

2 研究范围与方法

研究方法

本报告只使用本地一手证据或当前工作树中的实现文件，不依赖事后转述。核心证据分为四类：

- 运行时配置：对比故障期间保留下来的 compose 备份与修复后的 live compose。
- 运行时观测：`docker inspect`、`docker ps`、CCH 健康检查、host/container 网络探测。
- 应用日志：验证 provider 选择、是否走代理、以及成功请求是否已回到 direct path。
- 仓库实现：核对 deploy 脚本新增的 restart/probe 能力，以及 README 和单测是否同步。

2.1 证据边界

本报告区分两类结论：

- **强证据结论**：能直接落到备份文件、当前容器元数据、当前命令输出或 repo 文件。

- **次强结论**：例如 Docker NAT / `DOCKER` 链不一致，这在真实修复过程中确实被观察到，但当前没有像坏 `dns:` 一样保留下来的同等级原始证据。因此它在本报告中会被列为“可能放大故障的运行时副作用”，而不是最强根因。

2.2 时间表示

容器日志使用 UTC 时间戳，例如 `2026-03-25T12:29:18Z`；本地命令与 Docker service 观察使用 CST。为了避免“刚才”、“后来”这种模糊表述，正文尽量使用绝对日期和时间。

3 这次 Bug 到底坏在哪里

3.1 先排除一个常见误判：provider 没有被选错

如果问题真的出在 provider 配置本身，最典型的症状会是：

- provider selector 选不到 `RC/Hiyo`；
- 选到了 provider，但 endpoint 变成了错误地址；
- 请求还没到网络层就因为参数或 URL 拼接错误失败。

这次并不是这样。日志证据明确表明，在 `2026-03-25T12:29:18Z`，CCH 仍然做出了如下选择：

```
{"time":"2026-03-25T12:29:18.455Z","selected":{"name":"RC","id":1,"type":"codex"},"msg":"ProviderSelector: Selection decision"}
{"time":"2026-03-25T12:29:18.473Z","selectedEndpoints":[{"baseUrl":"https://right.codes/codex/"}],"msg":"ProxyForwarder: Trying provider"}
{"time":"2026-03-25T12:29:19.964Z","statusCode":200,"msg":"[ResponseHandler] Streaming request finalized as success"}
```

Listing 1: 应用日志显示 provider 选择仍然是 RC，并尝试命中 right.codes

判定

因此，“provider 被改爆”这个表述太宽了。更精确的说法是：**provider 配置层仍能正确选到 RC，真正出问题的是容器访问公网 endpoint 时依赖的解析与网络路径。**

3.2 最强根因：runtime compose 中的坏 DNS 覆盖

保留下来的备份文件 `docker-compose.archbox.yaml.bak-20260325-1912` 给出了最直接证据。对应片段如下：

```yaml
ports:
  - "${APP_PORT:-23000}:${APP_PORT:-23000}"
dns:
  - 119.29.29.29
  - 223.5.5.5
extra_hosts:
  - "host.docker.internal:host-gateway"
```

Listing 2: 故障期间 runtime compose 里的坏 DNS 覆盖

这意味着 app 容器不再按 Docker 默认方式使用宿主机提供的解析链，而是被强制指向两台公网 DNS。对一个需要稳定出公网、并且还要与宿主机代理桥接共存的容器来说，这是高风险改动，原因至少有三层：

- 它绕过了 Docker 在当前主机上的默认解析链，增加了环境耦合。
- 当网络策略、上游 DNS 路由、宿主机代理或 nftables/iptables 状态发生变化时，容器比宿主机更容易出现“宿主机能上网、容器却不稳定”的分裂。
- 这类改动不会改变 provider 名称或 URL，所以表面看起来像“某些 provider 失灵”，但根本层级实际上更低。

3.3 修复后的 live 容器已经不再携带该 DNS 覆盖

当前 live 容器的 `docker inspect` 结果显示：

```
null ["host.docker.internal:100.121.76.120"] {"com.docker.compose.project.config_files
    ":"/home/fanghaotian/Applications/claude-code-hub/docker-compose.archbox.yaml",
    ...}
```

Listing 3: 当前 live app 容器不再有 Docker 级别 Dns 覆盖

这里第一个字段就是 `HostConfig.Dns`，其值为 `null`。这说明当前 live CCH 容器已不再带着那段坏配置运行。同时，当前运行目录下的两个 compose 文件都已经清理掉了 `dns:` 块。

修复边界

修复不是“把 provider 改回去”，而是 **把运行时容器恢复到不带坏 DNS 覆盖的状态**。这也是为什么部署脚本后续需要增加 runtime probe，而不是只围绕 provider 表做修修补补。

4 为什么 Tailscale Serve 正常，不代表容器能出公网

这是这次讨论里最容易混淆的一点。下图把两条链路拆开了：

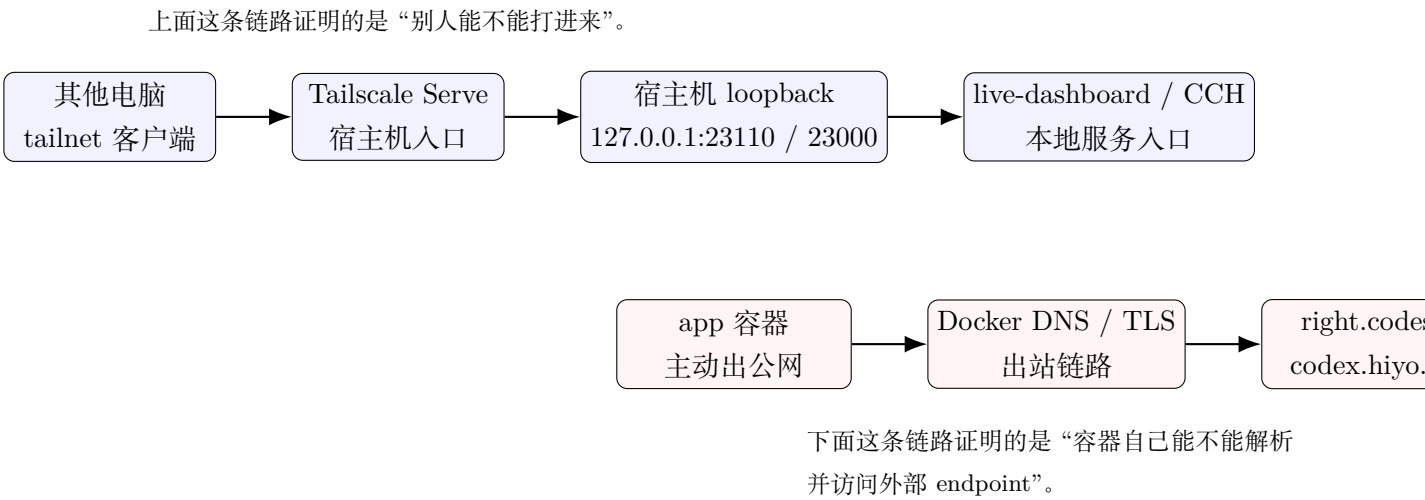

图 1: Tailscale Serve 的 ingress 路径与容器 egress 路径不是同一件事。

因此，“我看 Tailscale Serve 访问好像挺正常”这句话只证明：

- 宿主机上的 Serve 路由还在；
- 宿主机 loopback 端口能接到请求；
- 被转发到的本地服务入口没有完全死掉。

它并不能证明：

- app 容器的 DNS 解析链健康；
- 容器出站 TCP/TLS 正常；
- 某个外部 provider endpoint 的 TLS 行为正常。

5 当前状态的精确结论

5.1 `right.codes` 现在对 host 和容器都正常

当前命令输出显示：

- host 上执行 `curl -I -m 10 https://right.codes/` 返回 `HTTP/2 200`；
- live app 容器里执行 `curl -I -m 10 --http1.1 https://right.codes/` 返回 `HTTP/1.1 200 OK`；
- CCH 本地健康检查 `/api/actions/health` 返回 `{"status":"ok"}`；
- app 容器处于 `healthy` 状态。

这足以支持一个保守但明确的判断：**对 RC 这条路径，当前 live CCH 的容器 egress 已经恢复到可用状态**。

5.2 `codex.hiyo.top` 的现状是 endpoint 级残余，不是同一根因

当前 host 和容器里都能解析出 `codex.hiyo.top`，但二者都会得到同样的 TLS EOF：

```
curl: (35) TLS connect error: error:0A000126:SSL routines::unexpected eof while reading
```

Listing 4: 当前 host 与容器对 codex.hiyo.top 的共同症状

这里最重要的不是“它还失败”，而是 **host 和 container 失败得一样**。这会显著改变定位策略：

- 如果 host 能成、container 不能成，优先怀疑容器 DNS / egress；
- 如果 host 和 container 都同样失败，优先怀疑 endpoint 自身、TLS 协商、站点策略或外部网络条件。

不要把两个问题混成一个

“这次容器 DNS 覆盖导致 RC/Hiyo 无法直连”是主事故；“当前 `codex.hiyo.top` 在 host/container 同时 TLS EOF”是另一个更像 endpoint 侧的不稳定症状。后者不能反证前者没有发生，也不应该被误写成“CCH 现在还没修好 DNS”。

5.3 关于 Docker NAT / DOCKER 链

真实修复过程中，Docker NAT 状态的不一致确实让某次重启操作变得更糟，并促成了 daemon restart。但是，与坏 DNS 覆盖不同，本报告没有保留到同等级的一手证据片段。因此本报告只做谨慎结论：

- 它很可能是 **放大故障和增加恢复成本**的副问题；
- 但最强、最可复核的根因证据，仍然是运行时 compose 的坏 `dns:` 覆盖。

6 部署脚本加固了什么

6.1 bash 与 PowerShell 现在都支持 runtime 模式

这次对部署脚本的目标不是“让一键安装更漂亮”，而是把 **已有部署的运行态修复**做成可重复动作。对应实现位置：

- bash 参数与入口：`scripts/deploy.sh:66-94,135-182,1280-1282`
- bash runtime probe / restart 核心：`scripts/deploy.sh:500-848`
- bash 普通部署路径复用 probes：`scripts/deploy.sh:1123-1145,1305-1318`
- PowerShell 参数与入口：`scripts/deploy.ps1:80-108,115-169,1334-1339`
- PowerShell runtime probe / restart 核心：`scripts/deploy.ps1:400-860`
- PowerShell 普通部署路径复用 probes：`scripts/deploy.ps1:1164-1198,1363-1380`

6.2 新增能力总表

能力	现在做法	直接收益
已有部署重启	新增 `--restart` / `-Restart`，优先 `docker compose restart`，失败时回退到 `up -d`	把“先试 restart，不行再 up -d”变成脚本内建逻辑
纯探测模式	新增 `--probe-only` / `-ProbeOnly`	无需改容器即可先看现场状态，减少误操作
compose/env 显式定位	支持 `--compose-file`、`--env-file`、`--app-service`	适配 `archbox` 这类 runtime compose 不在仓库内的部署
宿主机探测	探测 `http://127.0.0.1:{APP_PORT}/api/actions/health`	直接判断本机入口是否可用
容器内探测	用 `docker exec` 在容器内打同一 health 路径	区分“宿主机端口坏”与“容器内部其实是活的”
运行态摘要	打印 `docker compose ps`、每个 service 的 state / health / ports	从“黑盒重启”升级到“带可见性的重启”
失败日志回显	不 ready 时自动打印最近日志 tail	避免每次故障都手敲多条命令

能力	现在做法	直接收益
Windows 对齐	PowerShell 版跟 bash 版保持基本一致的参数语义	文档和操作手册可以跨平台复用

6.3 README 和测试也同步了

- README 在 `README.md:167-183` 新增了“已有部署的重启与探测”小节，直接给出 Linux/macOS 与 Windows 示例。
- bash 侧新增单测 `tests/unit/scripts/deploy.test.ts:251-295`，覆盖两条最关键路径：
  - `probe-only` 会使用自定义 compose / env 并完成 host/container 探测；
  - `restart` 在 `docker compose restart` 失败时会回退到 `up -d`。

验证边界

bash 版本已通过 `bash -n` 和 Vitest 单测。PowerShell 版本因为当前工作机没有 `pwsh`，只能做到静态人工审阅，尚未本机执行验证。这一点应该在合并或交付时明确写给维护者，而不是默认当成已跑通。

7 以后怎么修

7.1 推荐的排障顺序

以后遇到“某个 provider 直连突然不行”，建议把排障顺序固定成下面这张表，而不是立刻改 provider 配置：

观察到的现象	host 探测	container 探测	优先动作
Tailscale Serve 正常，但 provider 请求失败	不足以判断	不足以判断	先跑 runtime probe，不要把 ingress 当 egress
host 能 `curl`，container 不能 `curl`	成功	失败	优先看 container DNS、compose 覆盖项、Docker egress
host 和 container 都失败，而且报错一致	失败	失败	优先怀疑 endpoint 自身、TLS 协商、上游网络条件
应用日志仍选中正确 provider，但报 `EAI_AGAIN`	可能成功也可能失败	可能失败	优先看 DNS / 容器网络层，不要先改 provider URL

观察到的现象	host 探测	container 探测	优先动作
日志出现 `Using proxy`	不相关	不相关	说明请求没有直连；先确认是否还残留 `proxy_url` 或代理桥接配置

7.2 推荐命令

```
./scripts/deploy.sh --probe-only -d /home/fanghaotian/Applications/claude-code-hub
curl -I -m 10 https://right.codes/
docker exec claude-code-hub-app-s6gz sh -lc 'curl␣-I␣-m␣10␣--http1.1␣https://right.
    codes/'
```

Listing 5: Linux/macOS 上建议的第一轮排障命令

```
.\scripts\deploy.ps1 -ProbeOnly -DeployDir C:\ProgramData\claude-code-hub
```

Listing 6: Windows 上建议的第一轮排障命令

7.3 真正要避免的不是“重启”，而是“盲重启”

这次事故暴露出来的最大流程问题，不是没人会重启，而是重启前后缺少统一的现场摘要：

- 不知道当前到底用了哪一个 compose 文件；
- 不知道 app service 的健康状态和端口映射；
- 不知道宿主机入口活不活；
- 不知道容器内部 health 活不活；
- 不知道失败后最近的日志应该先看哪里。

脚本加固的价值恰恰在这里：**先把现场抽象成固定的 probe，再决定是否重启**。

8 结论与后续问题

8.1 已被强证据支持的结论

- 本次主事故的最强根因是 runtime compose 中的坏 `dns:` 覆盖，而不是 provider base URL 被改错。
- Tailscale Serve 正常不能推出容器可以出公网，因为它验证的是 ingress，不是 egress。
- 当前 live CCH 容器已经不再带 Docker 级别 `Dns` 覆盖，CCH 健康检查正常，`right.codes` 从 host 和容器都可访问。
- bash 与 PowerShell 部署脚本已经新增 runtime probe / restart 能力，README 和 bash 单测也已同步。

8.2 大概率成立，但证据弱于主结论的判断

- Docker NAT / `DOCKER` 链的不一致在真实修复过程中确实增加了恢复复杂度，但它更像次级放大因素，而不是主事故的最佳解释。

8.3 仍待继续观察的问题

- `codex.hiyo.top` 当前在 host 和 container 都报 TLS EOF，这说明 Hiyo 端点现在还有一个与本次 DNS 事故不同的残余问题；如果业务仍依赖它，应该按 “endpoint/TLS 问题” 单独开案处理。
- `scripts/deploy.ps1` 需要在有 `pwsh` 的 Windows 机器上做一次真实运行验证，当前只能说实现已对齐、尚未本机执行。

A 来源清单

ID	模态	标题 / 标签	路径或说明
S1	配置备份	坏 DNS compose 备份	`/home/fanghaotian/Applications/claude-code-hub/docker-compose.archbox.yaml.bak-20260325-1912`
S2	配置	当前 runtime compose 状态	`/home/fanghaotian/Applications/claude-code-hub/docker-compose.archbox.yaml`；`/home/fanghaotian/Applications/claude-code-hub/docker-compose.yaml`
S3	运行时	live 容器元数据	`docker inspect claude-code-hub-app-s6gz --format ...`
S4	运行时	健康检查与容器状态	`docker compose ps`、`/api/actions/health`、`docker ps --format`
S5	日志	RC 请求日志	`docker logs claude-code-hub-app-s6gz`
S6	网络探测	host/container 连通性核验	`curl` 与 `docker exec ... curl`
S7	代码	bash deploy 脚本	`scripts/deploy.sh`
S8	代码	PowerShell deploy 脚本	`scripts/deploy.ps1`
S9	代码	bash 单测	`tests/unit/scripts/deploy.test.ts`
S10	文档	README restart/probe 入口	`README.md`
S11	备注	原始命令输出汇总	`docs/reports/2026-03-25-cch-network-postmortem/incident-evidence.md`